

Scraping web pages

When no API has the reviews you need, fetch the page's HTML and pull fields out of its tree with BeautifulSoup.

THE QUESTION

When no API has the data, how does a script pull movie fields out of a page built for human eyes?

THE INTUITION

A web page is nested tags that form a tree, the DOM. Scraping fetches that HTML, picks elements with a selector, reads their text or attributes, and turns each repeated block into one row.

KEY TAKEAWAY

Select the repeated card, then the fields inside it, and expect some to be missing. Check robots.txt, go slowly, and use an API when one exists.

robots.txt + ToS → requests: fetch HTML → BeautifulSoup: parse → select cards → text + attributes → rows → CSV

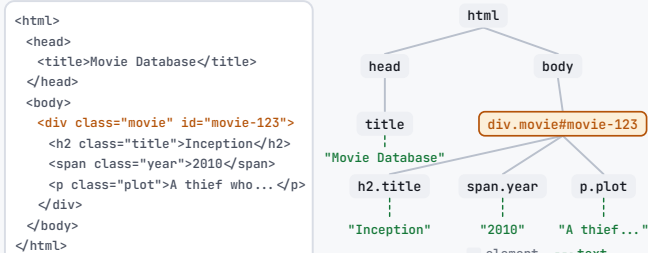
1 Scrape only when no dataset or API has it

- 1 ready-made dataset? yes → download it (Kaggle, HuggingFace)
- 2 official API? yes → call it: OMDb, TMDb (HTTP and APIs sheet)
- 3 scraping allowed? yes → check robots.txt + ToS, then scrape
- 4 none of these → by hand, partner with the owner, reframe

NetflixFlix plan: titles, ratings, genres from OMDb (API);
user reviews (IMDb), critic reviews (Rotten Tomatoes) by scraping

Scraping is plan B: a redesign breaks your selectors, HTML is unstructured, rate limits are unknown and the legal side is gray. First check DevTools; the page may load its data from a JSON API you can call directly (see the HTTP and APIs sheet).

2 HTML is a tree of nested tags



The browser parses HTML into the DOM (Document Object Model): each tag is a node, the tags inside it are its children, and text sits in the leaves. Scraping walks this tree. A class can repeat; an id should be unique. Right-click, Inspect shows the node behind anything on a page.

3 A tag has a name, attributes and text

```

<a href="/movies/123" class="btn">More info</a>

```

tag name	attribute	attribute	text
link.name	→ 'a'		
link["href"]	→ '/movies/123'		
link.get("href")	→ '/movies/123'		
link["title"]	→ KeyError (no such attribute)		
link.get("title")	→ None		
link.attrs	→ {'href': '/movies/123', 'class': ['btn']}		
link.text	→ 'More info'		

.get_text(strip=True) drops surrounding spaces and newlines. class comes back as a list because an element can have several. Use ["href"] when the attribute must exist and .get() when it may not; img["src"] gives an image's URL. .string is None when a tag holds more than one child; .text always works.

4 CSS selectors name what you want

selector	picks	matches
span	every element	4
.movie	class="movie"	2
#movie-1	id="movie-1" (unique)	1
div.movie	a div with class movie	2
.movie .title	.title anywhere inside .movie	2
span.rating	a span with class rating	2
a[href="/movies/"]	href starts with /movies/	2
a[href="/movies/matrix"]	href equals exactly	1

A dot means class, # means id, a space means "somewhere inside", [attr] tests an attribute (^= starts with). Counts are for the lab's page with two div.movie cards. DevTools can copy a selector; prefer short class-based ones, since long position-based paths break on a redesign.

5 find and select ask the same questions

find / find_all	select_one / select (CSS)
soup.h2	= select_one("h2")
find("h2", class_="title")	= select_one("h2.title")
find(id="movie-1")	= select_one("#movie-1")
find("a", href=True)	= select_one("a[href]")
find_all("div", class_="movie")	= select("div.movie")
find, select_one:	first match, or None
find_all, select:	a list, maybe empty []

Parse once with soup = BeautifulSoup(html, "html.parser"). Write class_, since class is a Python keyword. Search inside a result the same way: card.select_one(".year"). Move around with .parent and .children (text nodes have .name equal to None).

6 A table is rows of cells

tr	Movie	Year	Rating	
tr	Inception	2010	8.8	['Inception', 2010, 8.8]
tr	Interstellar	2014	8.6	'Interstellar', 2014, 8.6
tr	The Matrix	1999	8.7	'The Matrix', 1999, 8.7

th (header) · td (data)

```

for row in soup.find_all("tr")[1:]:
    td = row.find_all("td")
    movies.append({
        "title": td[0].text,
        "year": int(td[1].text),
        "rating": float(td[2].text)})

```

Cells are text; convert types yourself. pd.read_html(url) returns one DataFrame per <table> on the page.

Scrape many pages, politely

Loop over repeated cards, follow page numbers, survive missing tags, and respect robots.txt and the server.

7 One card becomes one row



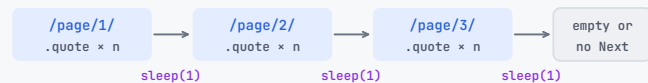
```
HEADERS = {"User-Agent":
            "MyBot/1.0 (contact@example.com)"}
```

```
def scrape_page(url):
    r = requests.get(url, headers=HEADERS,
                     timeout=10)
    r.raise_for_status()
    return BeautifulSoup(r.text,
                        "html.parser")
```

```
soup, movies = scrape_page(URL), []
for card in soup.select(".movie-card"):
    row = {"title": card.h2.text}
    for k in ("year", "genre", "rating"):
        el = card.select_one("." + k)
        row[k] = el.get_text(strip=True)
    movies.append(row)
```

Selecting inside each card keeps a film's fields together. Turning "8.8/10" into 8.8 is data cleaning, on the data validation sheet.

8 Follow the pages, with a pause



or follow the link: `soup.select_one("li.next a")["href"]`

```
BASE = "http://quotes.toscrape.com"
rows = []
for n in range(1, 4):          # 3 pages
    soup = scrape_page(f"{BASE}/page/{n}/")
    found = soup.select(".quote .text")
    if not found:
        break                  # ran out
    rows += [t.text for t in found]
    time.sleep(1)              # be polite
```

APIs page too: OMDb search gives 10 results a page; `totalResults` says how many page= calls.

12 Pause and think

Select all titles, then all ratings?

One card without a rating shifts the list onto the wrong films.

'NoneType' ... no attribute 'text'? A `select_one` matched nothing; test for `None` first.

`find("div", class="movie")` fails.

Why? `class` is a Python keyword, so this is a syntax error; write `class_`.

Why keep the raw HTML? A fixed selector re-parses saved pages; nothing is fetched twice.

Inspect shows cards, `select` gives `[]`? JavaScript added them; use the JSON call or Playwright.

Is `/search/about` allowed above? Yes: the longer Allow rule beats Disallow: `/search`.

Yes: the longer Allow rule beats Disallow: `/search`.

9 `select_one` returns `None` for a missing tag



```
el = card.select_one(".rating")
rating = el.text.strip() if el else None
```

Other common mistakes: no delay between requests; no try/except (log the failing URL, carry on); garbled accents (check `r.encoding`); not saving the raw HTML before parsing.

10 robots.txt says which paths bots may fetch

User-agent: *	✗ /search?q=inception	blocked
Disallow: /search	✓ /search/about	allowed
Allow: /search/about	✗ /?hl=en	blocked
Disallow: /?	✓ /maps?q=iitgn	allowed

Longest matching rule wins; no matching rule means allowed

Read `/robots.txt` and the Terms of Service before crawling (the lines above are from Google's). Name yourself in User-Agent, wait between requests, cache pages. Skip logins, personal data and heavy crawls.

11 JavaScript pages look empty to requests

requests / View Source	as sent	browser / Elements	after JS ran
<pre><body> <div id="app"></div> <script src="app.js"> </script> </body></pre>	<pre>select(".movie-card") → []</pre>	<pre><div id="app"> <div class="movie-card"> <h2>Inception</h2> ... <div class="movie-card"> ...</pre>	

requests never runs the page's scripts. If Inspect shows data that View Source lacks, call the JSON request from the Network tab, or drive a slower headless browser (Playwright, Selenium):

```
with sync_playwright() as p:
    page = p.chromium.launch().new_page()
    page.goto(URL)
    page.wait_for_selector(".movie-card")
    html = page.content()      # → BeautifulSoup
```

API first · select the card, then fields inside it · check every `select_one` for `None` · robots.txt, User-Agent, sleep(1)

Source: lectures "Data collection for machine learning" (web scraping part) and "HTML parsing", with their demos and lab, nipunbatra.github.io/stt-ai-teaching. HTTP, status codes and APIs are on the HTTP and APIs sheet.